What’s an agent?

Autonomous actions, ability to act

Ability to plan and execute any task

access to external tools, information

Behavior, actions, decision making

Observing the world and acting upon it

proactive in their approach to reaching their goals

Reasoning, Planning, Memory, Learning

Iteratively perform actions in the world

Reason about what it should do next to achieve its ultimate goal

Literature Search and Understanding

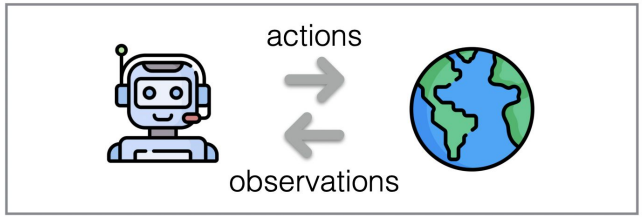
Ideation - generating “good” directions

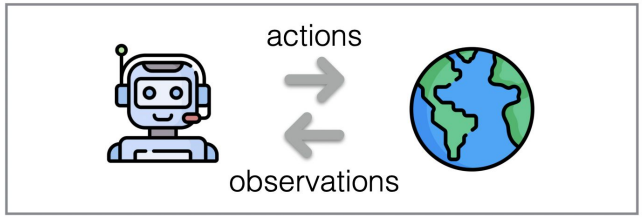
Forming and executing a research plan

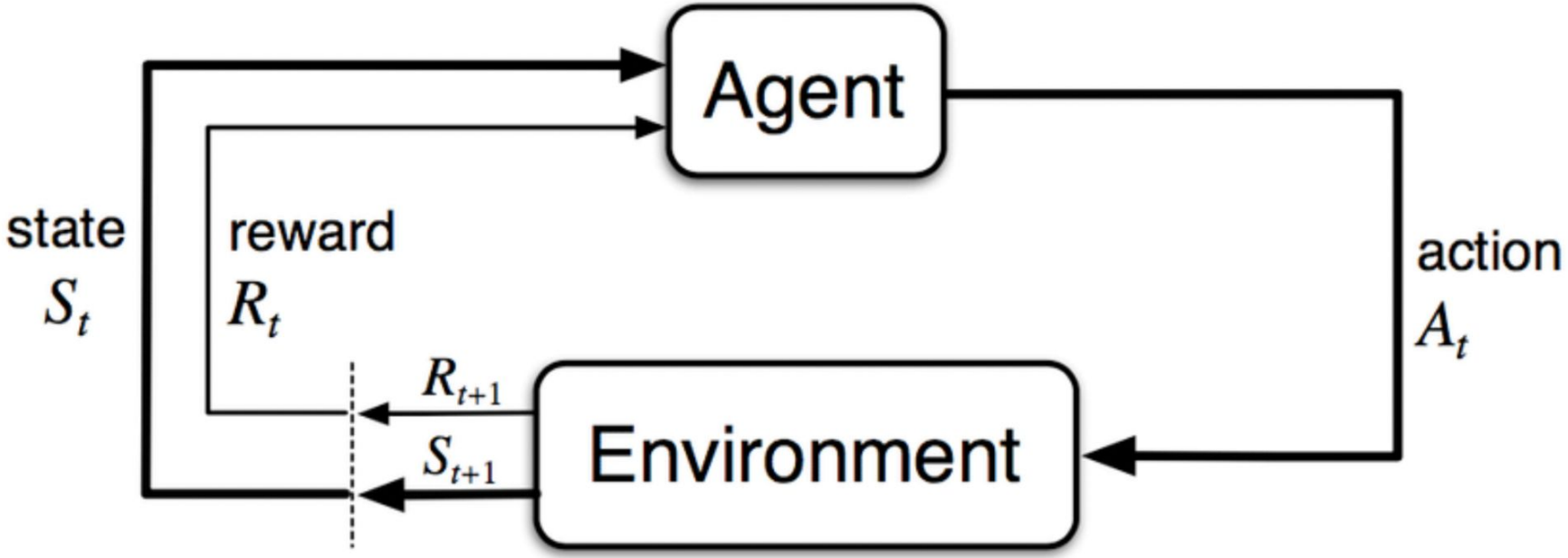
Data analysis

Critiquing quality, novelty

Drawing meaningful conclusions, insights

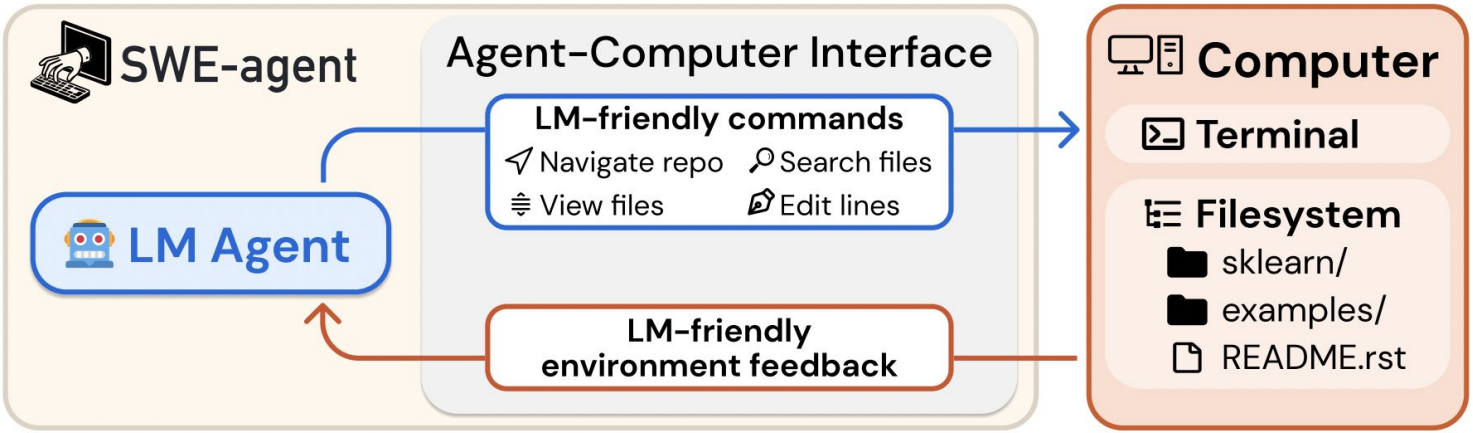

actions
observations


actions
observations


Agent
state
S_t
reward
R_t
action
A_t
R_{t+1}
S_{t+1}
Environment

Text agents

Text observations, actions



search_repo, find_file, goto line, navigate repo, open files, execute_code, edit lines…



You are standing in an open field west of a white house.
There is a small mailbox here.

> *open mailbox*

Opening the small mailbox reveals a leaflet.

> *read leaflet*

Welcome to Zork!

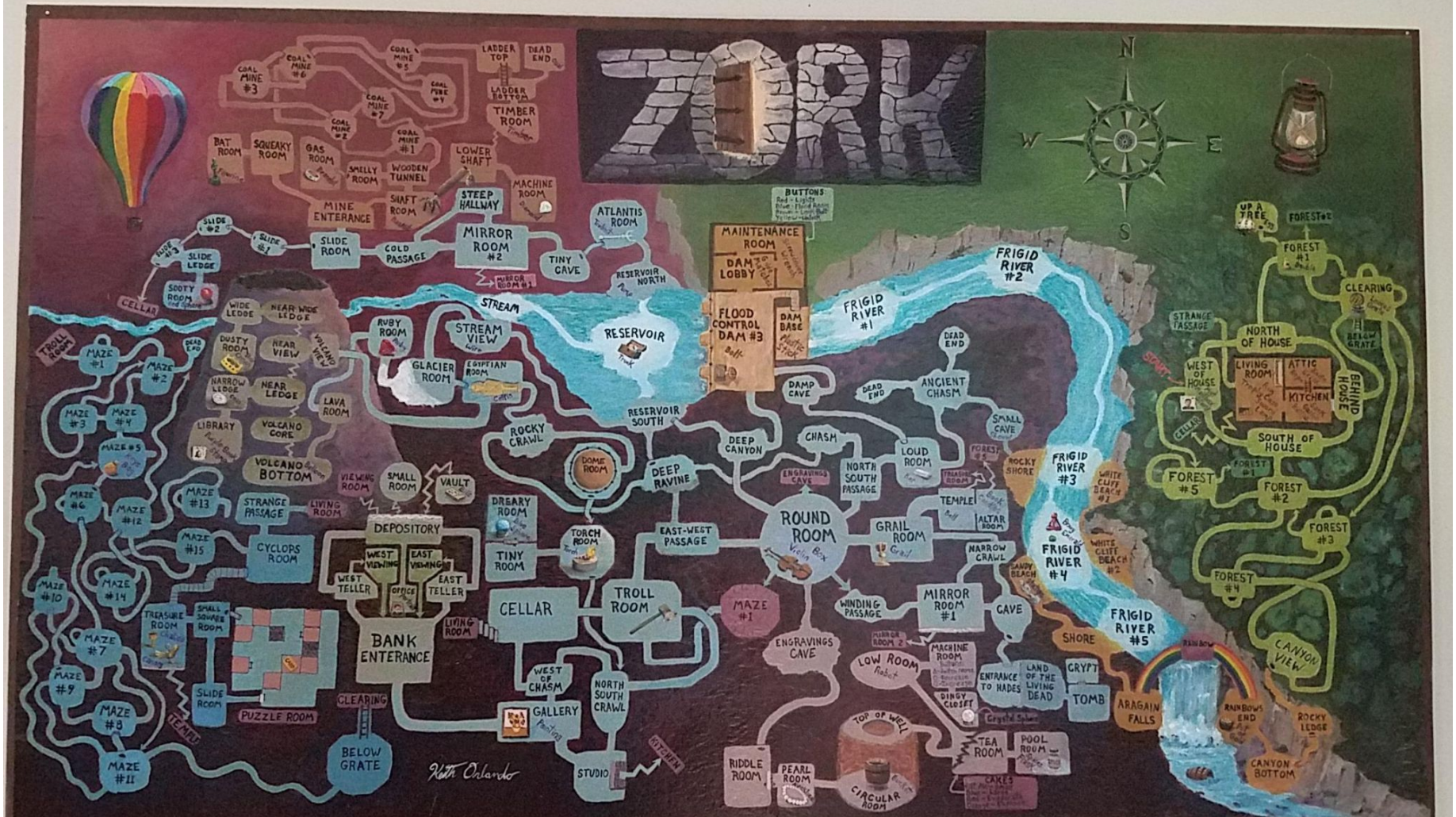
> *go north*

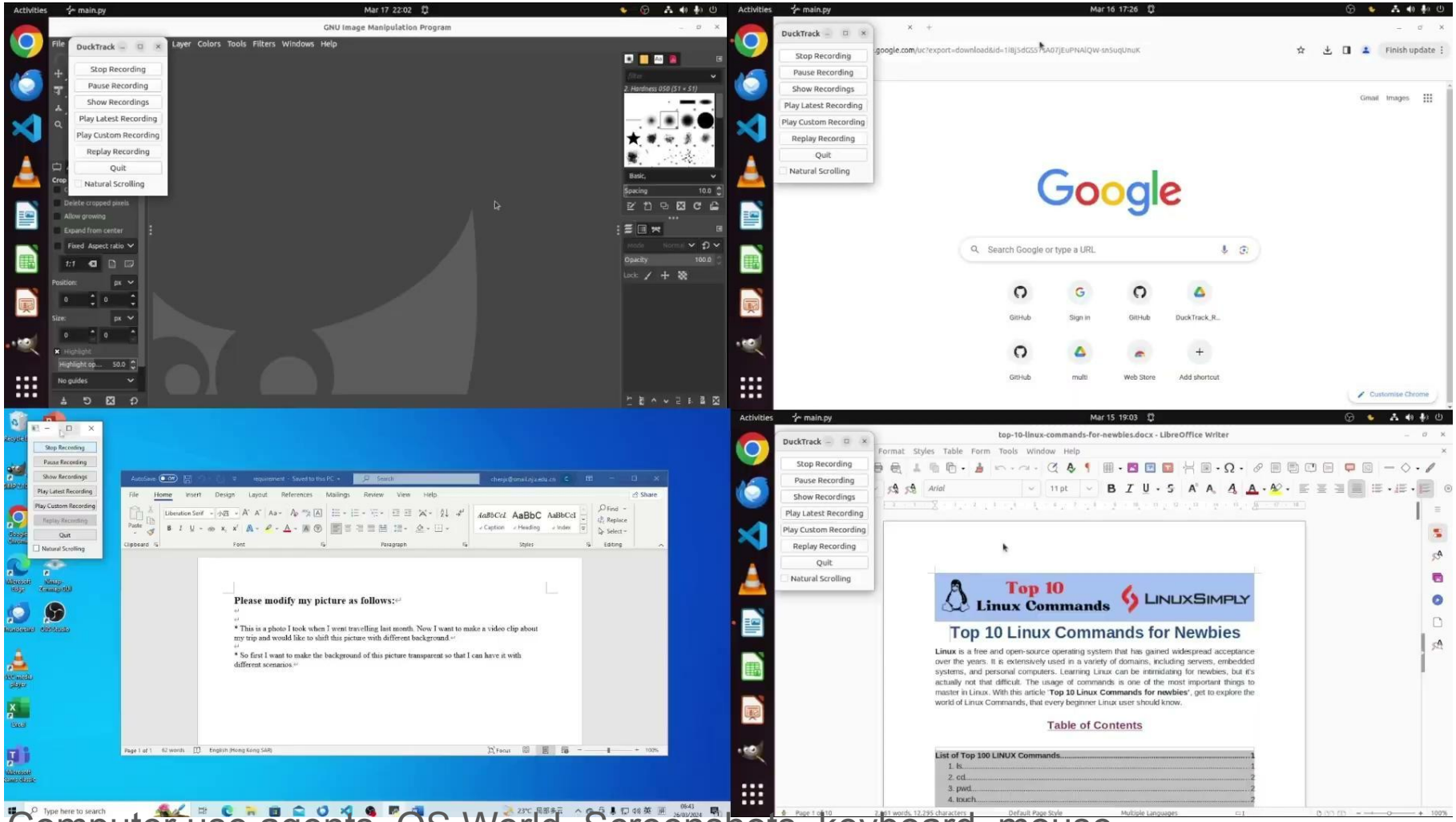
You are behind the white house. In one corner of the house there is a small window which is slightly ajar.

> *open window*

With effort you open the window far enough to allow entry.

> *enter house*


ZORK
N
W
E
S
COAL MINE #3
COAL MINE #4
COAL MINE #5
COAL MINE #6
COAL MINE #7
COAL MINE #8
COAL MINE #9
COAL MINE #10
LADDER TOP
DEAD END
LADDER BOTTOM
TIMBER ROOM
BAT ROOM
SQUEAKY ROOM
GAS ROOM
SMELLY ROOM
WOODEN TUNNEL
LOWER SHAFT
MINE ENTERANCE
SHAFT ROOM
STEEP HALLWAY
MACHINE ROOM
SLIDE #2
SLIDE #3
SLIDE #1
SLIDE ROOM
SLIDE LEDGE
COLD PASSAGE
MIRROR ROOM #2
MIRROR ROOM #1
TINY CAVE
ATLANTIS ROOM
CELLAR
SOOTY ROOM
STREAM
RESERVOIR NORTH
RESERVOIR
RESERVOIR SOUTH
BUTTONS
MAINTENANCE ROOM
DAM LOBBY
FLOOD CONTROL DAM #3
DAM BASE
FRIGID RIVER #1
FRIGID RIVER #2
DEAD END
ANCIENT CHASM
DAMP CAVE
SMALL CAVE
UP A TREE
FOREST
FOREST #1
CLEARING
BELOW GRATE
STRANGE PASSAGE
NORTH OF HOUSE
WEST OF HOUSE
LIVING ROOM
ATTIC
KITCHEN
BEHIND HOUSE
SOUTH OF HOUSE
FOREST #5
FOREST #2
FOREST #3
FOREST #4
CANYON VIEW
WIDE LEDGE
NEAR WIDE LEDGE
DUSTY ROOM
NEAR VIEW
VOLCANO VIEW
NARROW LEDGE
NEAR LEDGE
LAVA ROOM
LIBRARY
VOLCANO CORE
VOLCANO BOTTOM
RUBY ROOM
STREAM VIEW
GLACIER ROOM
EGYPTIAN ROOM
TROLL ROOM
MAZE #1
MAZE #2
MAZE #3
MAZE #4
MAZE #5
MAZE #6
MAZE #7
MAZE #8
MAZE #9
MAZE #10
MAZE #11
MAZE #12
MAZE #13
MAZE #14
MAZE #15
DEAD END
ROCKY CRAWL
DOME ROOM
DEEP CANYON
CHASM
ENGRAVING CAVE
NORTH SOUTH PASSAGE
LOUD ROOM
TEMPLE
ALTAR ROOM
TRADING ROOM
ROCKY SHORE
FRIGID RIVER #3
FRIGID RIVER #4
WHITE CLIFF BEACH #1
WHITE CLIFF BEACH #2
SANDY BEACH
DEEP RAVINE
ROUND ROOM
GRAIL ROOM
NARROW CRAWL
EAST-WEST PASSAGE
VIEWING ROOM
SMALL ROOM
VAULT
DEPOSITORY
WEST VIEWING
EAST VIEWING
WEST TELLER
OFFICE
EAST TELLER
STRANGE PASSAGE
LIVING ROOM
CYCLOPS ROOM
DREARY ROOM
TINY ROOM
TORCH ROOM
TREASURE ROOM
SMALL SEWER ROOM
BANK ENTERANCE
CELLAR
TROLL ROOM
MAZE #1
WINDING PASSAGE
MIRROR ROOM #1
CAVE
SHORE
FRIGID RIVER #5
ARAGAIN FALLS
RAINBOW
RAINBOW END
ROCKY LEDGE
CANYON BOTTOM
SLIDE ROOM
PUZZLE ROOM
TEMPLE
CLEARING
BELOW GRATE
WEST OF CHASM
GALLERY
NORTH SOUTH CRAWL
STUDIO
KITCHEN
ENGRAVINGS CAVE
LOW ROOM
MIRROR ROOM 2
MACHINE ROOM
DINGY CLOSET
ENTRANCE TO HADES
LAND OF THE LIVING DEAD
CRYPT
TOMB
TEA ROOM
POOL ROOM
CAKES
TOP OF WELL
CIRCULAR ROOM
RIDDLE ROOM
PEARL ROOM
Keith Orlando



Computer use agents. OS World. Screenshots, keyboard, mouse…



Select a scenario:

- Tutorial
- Combinatorial Chemistry
- Archaeology Dating
- Plant Nutrients
- Reactor Lab
- Lost in Translation
- Space Sick
- TODO 1
- TODO 2

Use the arrow keys to scroll between options, and press RETURN to select an option.

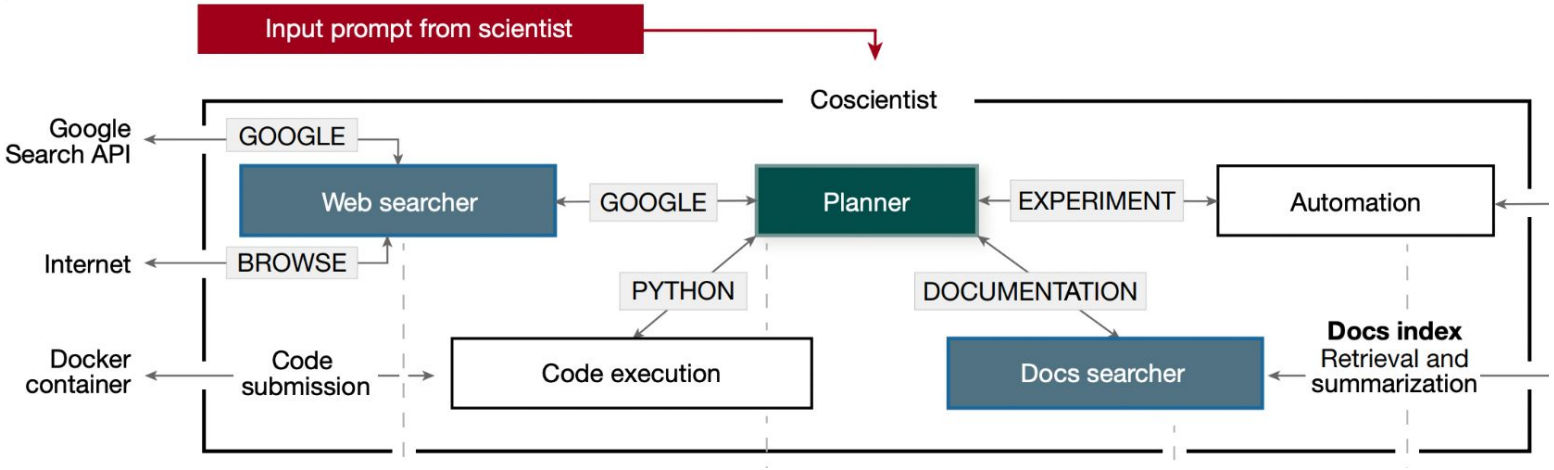
DiscoveryWorld is a virtual environment, in the form of a 2D top-down game, about benchmarking the ability for an agent to perform scientific discovery tasks.



Discovery World "Quantum Reactor" Scenario

- 8 high-level discovery tasks, with 10 variations of each,
- Easy/hard difficulty levels.
- Require using instruments.
- Many objects have physical properties (density, spectra)
- Requires data collection, deduction, statistics.
- Long horizon (100+ steps)

Scientific Agents for Laboratory Automation



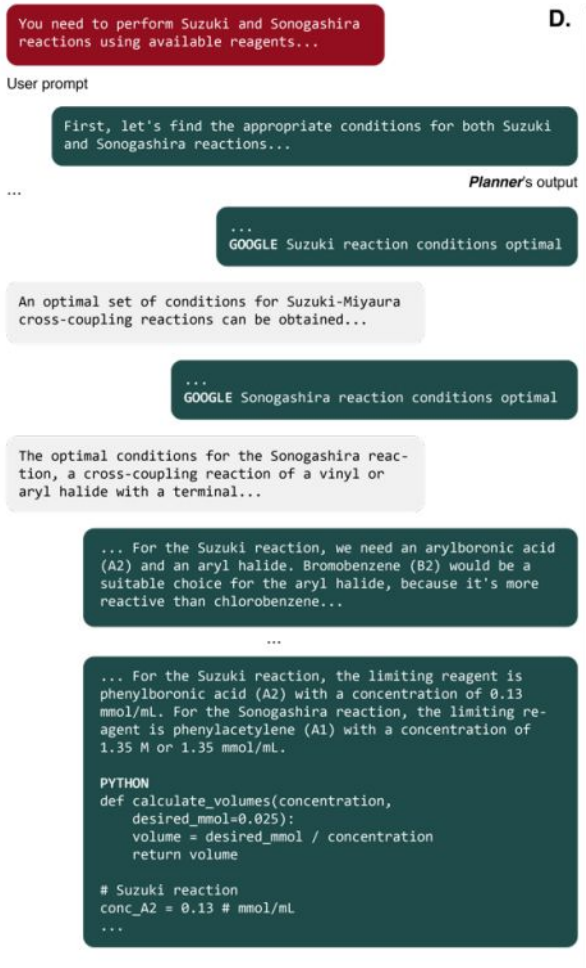
GPT-4 with tools to autonomously design, plan, and perform complex experiments.

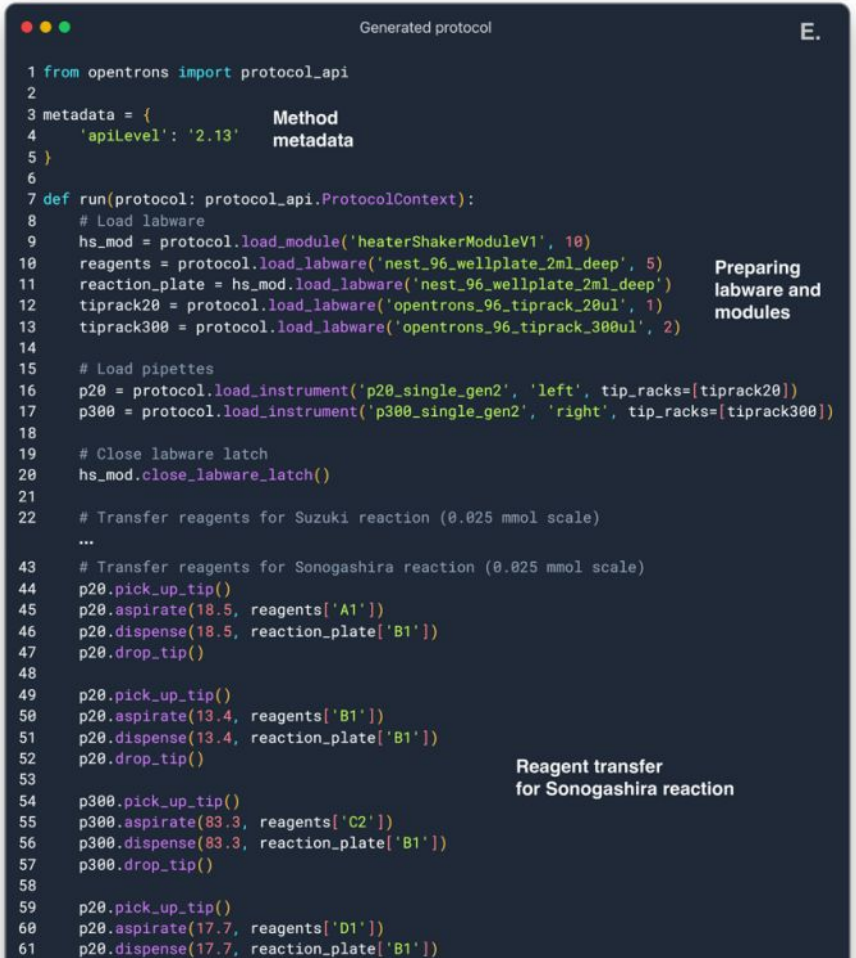
- Tools include internet and documentation search, code execution, experimental automation



* Autonomous chemical research with large language models, Nature, 2024

Generating an experimental protocol





LLMs for agents

LLMs for agents

LLM + instructions

LLMs for agents

LLM + instructions

Actions, list of tools

LLMs for agents

LLM + instructions

Actions, list of tools

Observations from environment (understand and represent environment)

LLMs for agents

LLM + instructions

Actions, list of tools

Observations from environment (understand and represent environment)

Interact with human users and other agents

LLMs for agents

LLM + instructions

Actions, list of tools

Observations from environment (understand and represent environment)

Interact with human users and other agents

Reasoning and planning

Reason + Act (ReAct)

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao*[,1], Jeffrey Zhao[2], Dian Yu[2], Nan Du[2], Izhak Shafran[2], Karthik Narasimhan[1], Yuan Cao[2]

[1]Department of Computer Science, Princeton University
[2]Google Research, Brain team
[1]{shunyuy,karthikn}@princeton.edu
[2]{jeffreyzhao,dianyu,dunan,izhak,yuancao}@google.com

arXiv:2210.03629v3 [cs.CL] 10 Mar 2023

ABSTRACT

While large language models (LLMs) have demonstrated impressive performance across tasks in language understanding and interactive decision making, their abilities for reasoning (e.g. chain-of-thought prompting) and acting (e.g. action plan generation) have primarily been studied as separate topics. In this paper, we explore the use of LLMs to generate both reasoning traces and task-specific actions in an interleaved manner, allowing for greater synergy between the two: reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while actions allow it to interface with and gather additional information from external sources such as knowledge bases or environments. We apply our approach, named `ReAct`, to a diverse set of language and decision making tasks and demonstrate its effectiveness over state-of-the-art baselines in addition to improved human interpretability and trustworthiness. Concretely, on question answering (HotpotQA) and fact verification (Fever), `ReAct` overcomes prevalent issues of hallucination and error propagation in chain-of-thought reasoning by interacting with a simple Wikipedia API, and generating human-like task-solving trajectories that are more interpretable than baselines without reasoning traces. Furthermore, on two interactive decision making benchmarks (ALFWorld and WebShop), `ReAct` outperforms imitation and reinforcement learning methods by an absolute success rate of 34% and 10% respectively, while being prompted with only one or two in-context examples.

1 INTRODUCTION

A unique feature of human intelligence is the ability to seamlessly combine task-oriented actions with

Thought → Action → Observation

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod ✗

(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch ✗

(1c) Act-Only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control …

Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: …

Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Act 4: Finish[yes] ✗

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple … originally designed to control the Front Row media center program …

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports',' Front Row (software) ', …]

Thought 3: Front Row is not found. I need to search Front Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys] ✓

ReAct Pseudo Code

```
MAX_STEPS = 10
for step in range(0, MAX_STEPS):

   # Get observation from the environment
   observation = env.getObservation()

   # Do the ReAct think-then-act 2-stage prompt
   think_response = send_think_prompt_llm()
   act_response = send_act_prompt_llm(think_response)
   action = parse_action_from_response(act_response)

   # Perform the action
   score, reward, is_completed = env.sendAction(action)
   if (is_completed == true):
      break
```



"MEZE is a memory-efficient zeroth-order optimizer. Train with this optimizer on my dataset (available at https://drive...), following the paper's code at: https://github.com/... and report accuracy."

...

I need to install the required dependencies listed in the requirements.txt file

```
!pip install -r requirements.txt
```

```
Successfully installed datasets-2.19.2 transformers-4.40.2 ...
```

I will now download the provided data file.

```
!gdown.download("https://drive.google.com../", "my_data.csv")
```

```
Downloading to: /content/dpt/my_data.csv
```

Next, I will edit the `data_utils.py` file to configure the loaded dataset.

```
edit data_utils.py <BEFORE>
        return datasets.load_dataset('glue', 'mrpc', split=split)
<AFTER>
        return datasets.load_dataset('csv', data_files={...
```

```
File edited successfully.
```

Setting up the experiment

I will now execute the experiment with the `run.py` script.

```
!python run.py output_dir result/output ...
```

```
Traceback (most recent call last): ...
ImportError: cannot import name 'default_hp_search_backend'
```

Error suggests a `transformers` version issue. I will downgrade it to resolve the error.

```
!pip install transformers==4.28.1  # downgrading dependency
```

```
Successfully installed tokenizers-0.13.3 transformers-4.28.1
```

Resolving Issues

...

I will now retry running the experiment.

```
!python run.py output_dir result/output ...
```

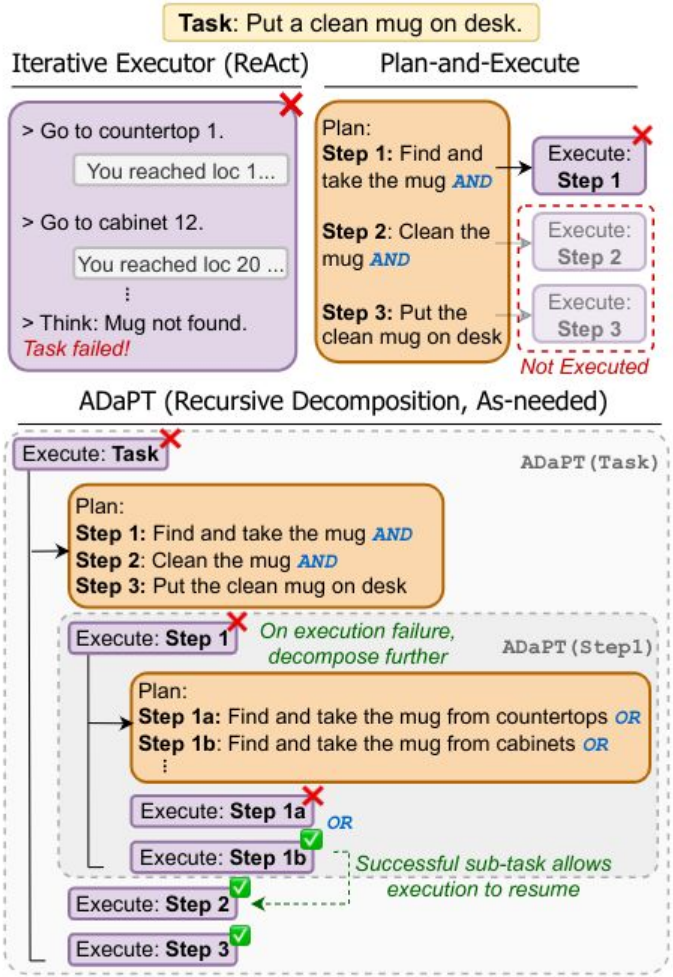
```
18:50:24,975 - INFO - ***** Running training *****
...
18:51:12,059 - INFO - {'dev_accuracy': 0.27}
```

Executing & Reporting metrics

The training process has completed successfully. The accuracy result on the development set is: **0.27**

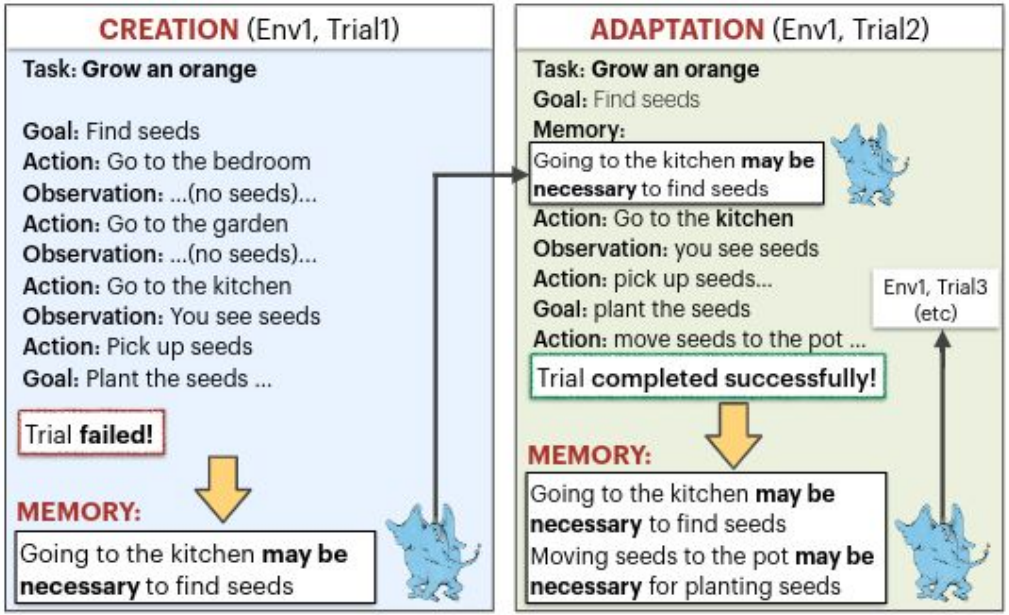
SUPER: Evaluating Agents on Setting Up and Executing Tasks from Research Repositories

- Iteratively **decompose** larger tasks into smaller subtasks
- Generates plans for subtasks
- If the plan fails, it tries to recover and generate a new decomposition/plan **as needed**



ADAPT: As-Needed Decomposition and Planning with Language Models

- Agent tries a game, reflects on its performance, then builds a **memory** of “lessons” that it can use the next time it tries.
- A continual-learning paradigm



CLIN for Rapid Task Adaptation

The more you try a task, the more lessons you'll learn from it.

The more lessons you write down in memory, the better you'll do next time you try that task.

Executable Code Actions Elicit Better LLM Agents

Instruction: Determine the most cost-effective country to purchase the smartphone model "CodeAct 1". The countries to consider are the USA, Japan, Germany, and India.

Available APIs
[1] lookup_rates(country: str) -> (float, float)
[2] convert_and_tax(price: float, exchange_rate: float, tax_rate: float) -> float
[3] estimate_final_price(converted_price: float, shipping_cost: float) -> float
[4] lookup_phone_price(model: str, country: str) -> float
[5] estimate_shipping_cost(destination_country: str) -> float

LLM Agent using [**Text/JSON**] as Action

Think I should calculate the phone price in USD for each country, then find the most cost-effective country.

Action
```
Text: lookup_rates, Germany
JSON: {"tool": "lookup_rates", "country": "Germany"}
```

Environment 1.1, 0.19

Action
```
Text: lookup_phone_price, CodeAct 1, Germany
JSON: {"tool": "lookup_phone_price", "model": "CodeAct 1", "country": "Germany"}
```

Environment 700

Action
```
Text: convert_and_tax, 700, 1.1, 0.19
JSON: {"tool": "convert_and_tax", "price": 700, "exchange_rate": 1.1, "tax_rate": 0.19}
```

Environment 916.3

[... interactions omitted (look up shipping cost and calculate final price) ...]

Action
```
Text: lookup_rates, Japan
JSON: {"tool": "lookup_rates", "country": "Japan"}
```

[... interactions omitted (calculate final price for all other countries)...]

Response The most cost-effective country to purchase the smartphone model is Japan with price 904.00 in USD.

CodeAct: LLM Agent using [**Code**] as Action

Think I should calculate the phone price in USD for each country, then find the most cost-effective country.

Action

```python
countries = ['USA', 'Japan', 'Germany', 'India']
final_prices = {}

for country in countries:
    exchange_rate, tax_rate = lookup_rates(country)
    local_price = lookup_phone_price("xAct 1", country)
    converted_price = convert_and_tax(
      local_price, exchange_rate, tax_rate
    )
    shipping_cost = estimate_shipping_cost(country)
    final_price = estimate_final_price(converted_price, shipping_cost)
    final_prices[country] = final_price

most_cost_effective_country = min(final_prices, key=final_prices.get)
most_cost_effective_price = final_prices[most_cost_effective_country]
print(most_cost_effective_country, most_cost_effective_price)
```

Control & Data Flow of Code Simplifies Complex Operations

Re-use `min` Function from Existing Software Infrastructures (Python library)

Environment 1.1, 0.19

Response The most cost-effective country to purchase the smartphone model is Japan with price 904.00 in USD.

> Fewer Actions Required!

- Actions/tools - names, what they do, how access
- Observation and output format - description, examples
- Memory - what agent done/seen so far. Short-term vs long term.

- Actions/tools - names, what they do, how access
- Observation and output format - description, examples
- Memory - what agent done/seen so far. Short-term vs long term.

- Iterative incorporation of feedback
  - Collect feedback - e.g., was action successful? Was user satisfied?
  - Integrate new information into its existing context/memory.
  - Adapts strategies based on new info

- **Actions/tools - names, what they do, how access**

- Observation and output format - description, examples

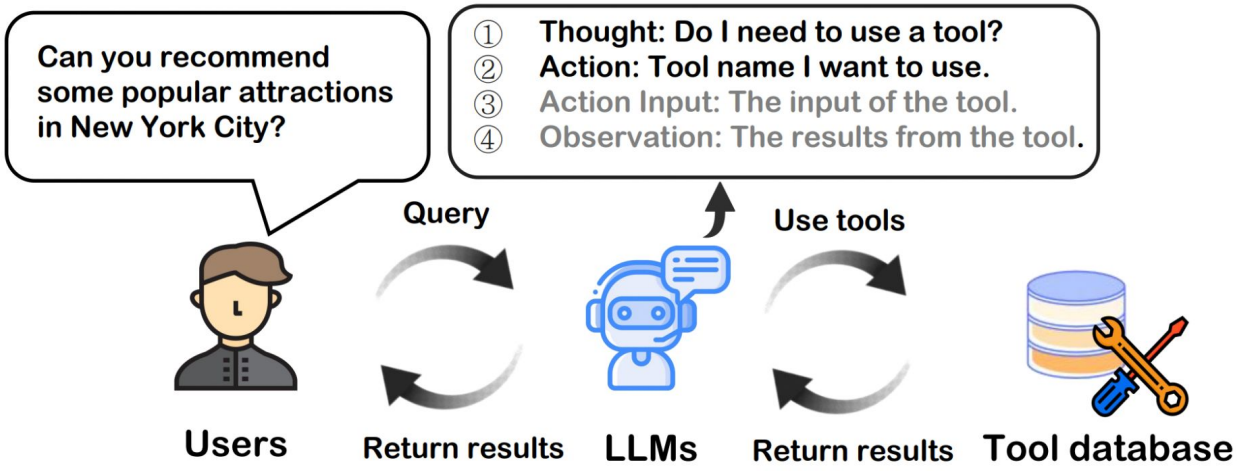
- Memory - what agent done/seen so far. Short-term vs long term.

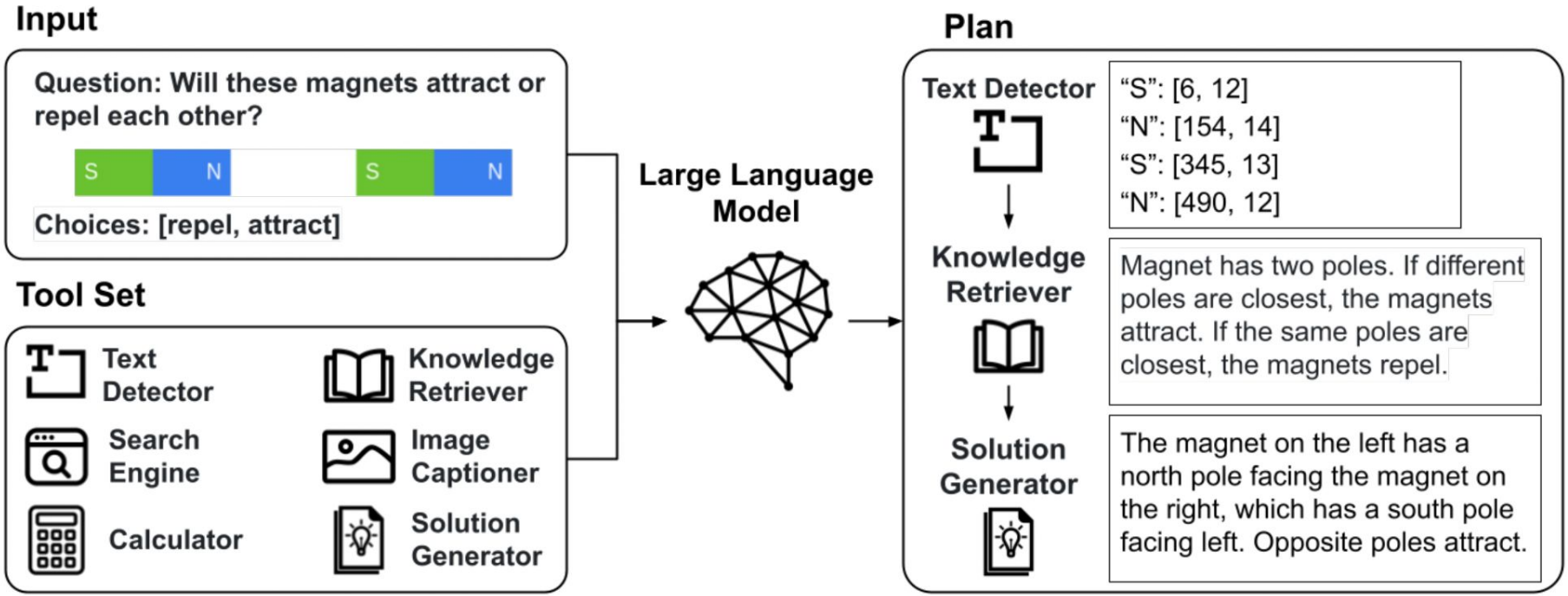
- Iterative incorporation of feedback
  - Collect feedback - e.g., was action successful? Was user satisfied?
  - Integrate new information into its existing context/memory.
  - Adapts strategies based on new info

Tool use

Tools - think of API

API – input (actions), output (observations)


Can you recommend some popular attractions in New York City?
① Thought: Do I need to use a tool?
② Action: Tool name I want to use.
③ Action Input: The input of the tool.
④ Observation: The results from the tool.
Query
Use tools
Users
Return results
LLMs
Return results
Tool database


Input
Question: Will these magnets attract or repel each other?
S
N
S
N
Choices: [repel, attract]
Tool Set
Text Detector
Knowledge Retriever
Search Engine
Image Captioner
Calculator
Solution Generator
Large Language Model
Plan
Text Detector
"S": [6, 12]
"N": [154, 14]
"S": [345, 13]
"N": [490, 12]
Knowledge Retriever
Magnet has two poles. If different poles are closest, the magnets attract. If the same poles are closest, the magnets repel.
Solution Generator
The magnet on the left has a north pole facing the magnet on the right, which has a south pole facing left. Opposite poles attract.

Chemcrow

nature machine intelligence

Article

https://doi.org/10.1038/

Augmenting large language models wi chemistry tools

Received: 13 September 2023

Accepted: 27 March 2024

Published online: 8 May 2024

Andres M. Bran[1,2,6], Sam Cox[3,4,6], Oliver Schilter[1,2,5], Carlo Ba
Andrew D. White[3,4] & Philippe Schwaller[1,2]

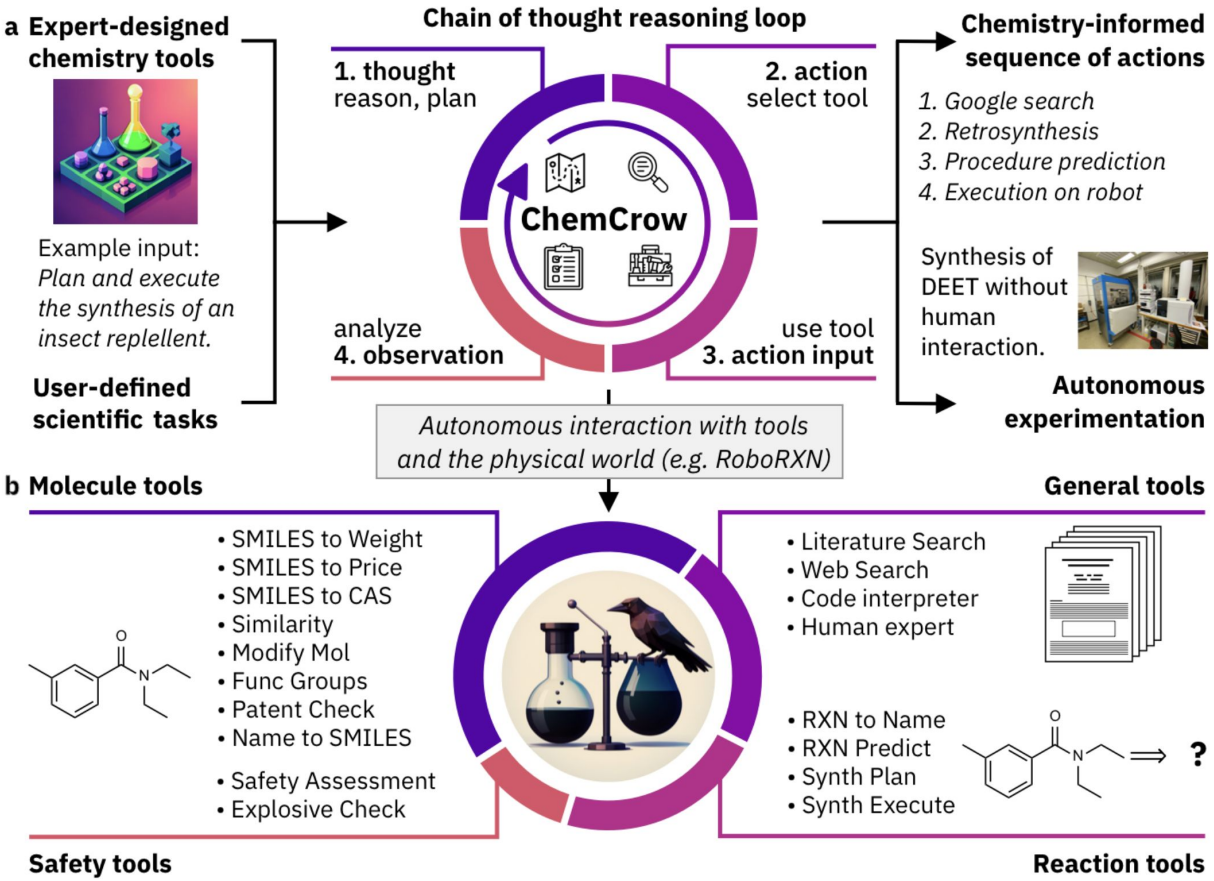
Crows surprise researchers with advanced tool use

New Caledonian crows are proving they are not bird-brained by using tools not just to find food, but also to assess danger



Abel, one of the first New Caledonian crows at Oxford University to demonstrate tool use. Photograph: Alex Weir/Oxford University

- Chemcrow uses tools for chemical reasoning
  - This covers organic synthesis, drug discovery, and materials design
- Use an iterative, action-based pipeline of tools (ReAct)




IBM RoboRXN for Chemistry
IBM Research Europe in Zurich

```python
from chemcrow.agents import ChemCrow

# Initialize ChemCrow
chem_model = ChemCrow(
  model="gpt-4-0613",
  temp=0.1,
  verbose=True
)

# Define task
task = (
  "Find and synthesize a thiourea organocatalyst "
  "which accelerates a Diels-Alder reaction."
)

# Execute ChemCrow
chem_model.run(task)
```

b **Task:** Find and synthesize a thiourea organocatalyst which accelerates a Diels-Alder reaction.

First, I need to find a thiourea organocatalyst that can accelerate the Diels-Alder reaction. I will perform a web search to find a suitable catalyst.
Web Search tool: Schreiner's thiourea catalyst

Now, I will obtain the SMILES. **Name2Smiles tool:**
FC(F)(F)c1cc(NC(=S)Nc2cc(C(F)(F)F)cc(C(F)(F)F)c2)cc(C(F)(F)F)c1

I will plan a synthesis for Schreiner's thiourea catalyst.
SynthesisPlanner tool: detailed synthesis plan

I will execute the synthesis.
SynthesisExecuter tool: successful synthesis.

Connection with physical world →

c **RoboRXN synthesis platform**



1. The LLM is provided with a list of tool names, descriptions of their utility and details about the expected input/output.
2. It is then instructed to answer a user-given prompt, using the tools provided when necessary.

1. The LLM is provided with a list of tool names, descriptions of their utility and details about the expected input/output.
2. It is then instructed to answer a user-given prompt, using the tools provided when necessary.
3. The model is guided to follow the **Thought, Action, Action Input, Observation** format, which requires it to
    a. reason about the current state of the task,
    b. consider its relevance to the final goal,
    c. plan the next steps accordingly

1. The LLM is provided with a list of tool names, descriptions of their utility and details about the expected input/output.
2. It is then instructed to answer a user-given prompt, using the tools provided when necessary.
3. The model is guided to follow the **Thought, Action, Action Input, Observation** format, which requires it to
    a. reason about the current state of the task,
    b. consider its relevance to the final goal,
    c. plan the next steps accordingly
4. After the reasoning in the Thought step, the LLM requests a tool (preceded by the keyword ‘Action’) and the input for this tool (with the keyword ‘Action Input’).

1. The LLM is provided with a list of tool names, descriptions of their utility and details about the expected input/output.
2. It is then instructed to answer a user-given prompt, using the tools provided when necessary.
3. The model is guided to follow the **Thought, Action, Action Input, Observation** format, which requires it to
    a. reason about the current state of the task,
    b. consider its relevance to the final goal,
    c. plan the next steps accordingly
4. After the reasoning in the Thought step, the LLM requests a tool (preceded by the keyword ‘Action’) and the input for this tool (with the keyword ‘Action Input’).
5. The text generation then pauses, and the program attempts to execute the requested function using the provided input. The result is returned to the LLM prepended by the keyword ‘Observation’, and the LLM proceeds to the Thought step again. It continues iteratively until the final answer is reached.

What is a tool

- Basically, a **function**
- What the tool does + What inputs it expects
  a. A textual description of what the function does.
  b. The function’s arguments (better if with typings).

```
tools=[
 {
  "type": "function",
  "function": {
      "name": "get_weather",
      "description": "Get current temperature for a given location.",
      "parameters": {
          "type": "object",
          "properties": {
              "location": {
                  "type": "string",
                  "description": "City and country e.g. Bogotá, Colombia"
              }
          },
          "required": [
              "location"
          ],
          "additionalProperties": False
      },
      "strict": True
  }
},
```

```
    {
      "type": "function",
      "function": {
        "name": "get_rain_probability",
        "description": "Get the probability of rain for a specific location",
        "parameters": {
          "type": "object",
          "properties": {
            "location": {
              "type": "string",
              "description": "The city and state, e.g., San Francisco, CA"
            }
          },
          "required": ["location"]
        }
      }
    }
  ]
)
```

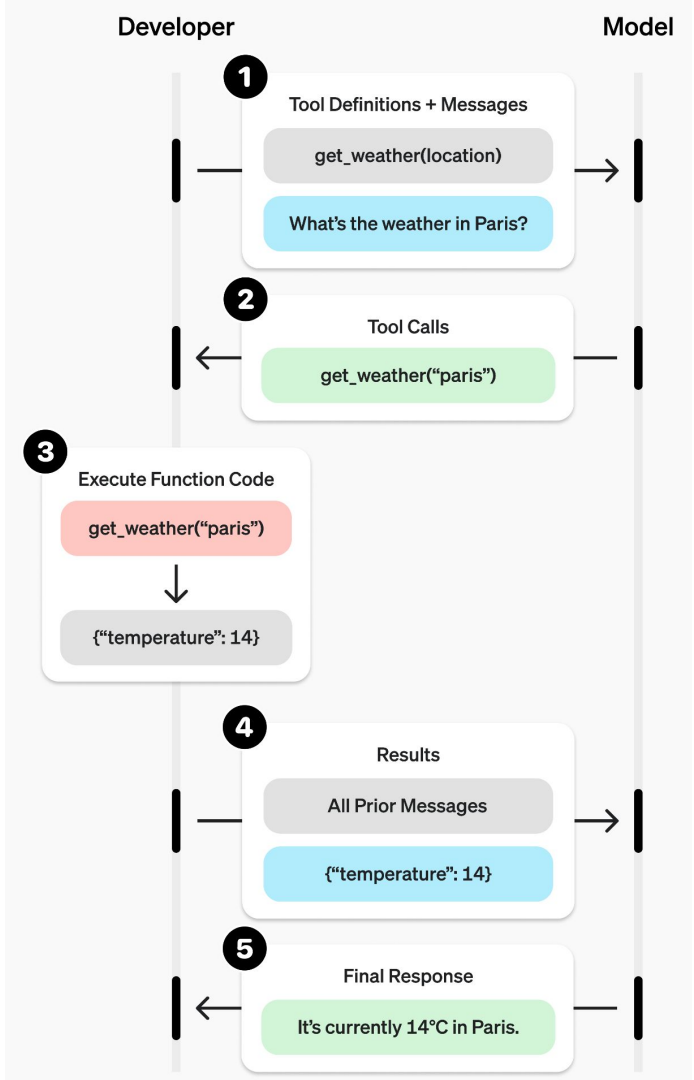
```python
messages = [{"role": "user", "content": "What's the weather like in Paris today?"}]

completion = client.chat.completions.create(
    model="gpt-4o",
    messages=messages,
    tools=tools,
)
```

```python
print(completion.choices[0].message.tool_calls)
```

```
[{
    "id": "call_12345xyz",
    "type": "function",
    "function": {
        "name": "get_weather",
        "arguments": "{\"location\":\"Paris, France\"}"
    }
}]
```


Call to some weather web API, returns temp, then LLM reads tool call output and returns final NL answer


Developer
Model
1
Tool Definitions + Messages
get_weather(location)
What's the weather in Paris?
2
Tool Calls
get_weather("paris")
3
Execute Function Code
get_weather("paris")
{"temperature": 14}
4
Results
All Prior Messages
{"temperature": 14}
5
Final Response
It's currently 14°C in Paris.

[{'type': 'function', 'function': {'name': **'search_pubmed'**, 'description'**: 'Search for scientific articles in PubMed database',** 'parameters': {'type': 'object', 'properties': {'query': {'type': 'string', 'description': 'The search query for PubMed'}, 'max_results': {'type': 'integer', 'description': 'Maximum number of results to return (default: 5)'}, 'sort_by': {'type': 'string', 'enum': ['relevance', 'date', 'citations'], 'description': 'How to sort the results'}}, 'required': ['query']}}},

{'type': 'function', 'function': {**'name'**: **'analyze_molecule'**, 'description': '**Analyze molecular structure and properties**', 'parameters': {'type': 'object', 'properties': {'molecule_id': {'type': 'string', 'description': 'Molecule identifier (PubChem CID, SMILES, or InChI)'}, 'properties': {'type': 'array', 'items': {'type': 'string', 'enum': ['3d_structure', 'pharmacokinetics', 'binding_affinity', 'toxicity']}, 'description': 'Properties to analyze'}}, 'required': ['molecule_id']}}},

{'type': 'function', 'function': {'name': **'analyze_dataset'**, 'description': '**Perform statistical analysis on scientific datasets**', 'parameters': {'type': 'object', 'properties': {'dataset_url': {'type': 'string', 'description': 'URL of the dataset to analyze'}, 'analysis_type': {'type': 'string', 'enum': ['descriptive', 'regression', 'correlation', 'clustering'], 'description': 'Type of analysis to perform'}}, 'required': ['dataset_url', 'analysis_type']}}}]

{ "from": "system", "value": "You are a function calling AI model. You are provided with function signatures within <tools> </tools> XML tags. You may call one or more functions to assist with the user query. Don't make assumptions about what values to plug into functions.\n

<tools>\n

[

{'type': 'function', 'function': {'name': '**simulate_quantum_entanglement**', 'description': 'Simulates the process of quantum entanglement generation, distribution, and verification between two distant quantum nodes, including the effects of atmospheric and diffraction losses.', 'parameters': {'type': 'object', 'properties': {'node_distance': {'type': 'string', 'description': 'The distance in kilometers between the two quantum nodes.'}, 'include_atmospheric_loss': {'type': 'string', 'description': 'Flag to include atmospheric loss in the simulation.'}, 'include_diffraction_loss': {'type': 'string', 'description': 'Flag to include diffraction loss in the simulation.'}}, 'required': ['node_distance', 'include_atmospheric_loss', 'include_diffraction_loss']}}},

{'type': 'function', 'function': {'name': '**calculate_quantum_channel_capacity**', 'description': 'Calculates the quantum channel capacity for a given distance and set of environmental factors.', 'parameters': {'type': 'object', 'properties': {'node_distance': {'type': 'string', 'description': 'The distance in kilometers between the two quantum nodes.'}, 'environmental_factors': {'type': 'object', 'description': 'A set of environmental factors such as atmospheric conditions, temperature, and other relevant parameters.'}}, 'required': ['node_distance', 'environmental_factors']}}},

{'type': 'function', 'function': {'name': '**verify_quantum_entanglement**', 'description': 'Verifies the presence and quality of quantum entanglement between two nodes after distribution.', 'parameters': {'type': 'object', 'properties': {'entanglement_data': {'type': 'string', 'description': 'The data obtained from the entanglement distribution simulation.'}}, 'required': ['entanglement_data']}}}

]\n

</tools>\n

Generate synthetic data for fine-tuning example

https://cookbook.openai.com/examples/fine_tuning_for_function_calling

Learning to use tools

ICL

Demonstration

Description: examples of questions and the tool-use plan.

- **Question:**
  Which property do these objects have in common?



  Tool-use Plan:

  Text Detector → Knowledge Retriever → Solution Generator

- **Question:** […]

  Tool-use Plan: […]

- **Question:** […]

  Tool-use Plan: […]



Documentation

Description: available tools and their functionalities.

- **Text Detector:**
  It detects the text in an image […]
- **Knowledge Retriever:**
  It retrieves relevant knowledge […]
- **Search Engine:**
  It searches the web for relevant info […]
- **Image Captioner:**
  It generates a caption for an image […]
- …





Your task is to add calls to a Question Answering API to a piece of text. The questions should help you get information required to complete the text. You can call the API by writing "[QA(question)]" where "question" is the question you want to ask. Here are some examples of API calls:

Input: Joe Biden was born in Scranton, Pennsylvania.

Output: Joe Biden was born in [QA("Where was Joe Biden born?")] Scranton, [QA("In which state is Scranton?")] Pennsylvania.

Input: Coca-Cola, or Coke, is a carbonated soft drink manufactured by the Coca-Cola Company.

Output: Coca-Cola, or [QA("What other name is Coca-Cola known by?")] Coke, is a carbonated soft drink manufactured by [QA("Who manufactures Coca-Cola?")] the Coca-Cola Company.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from “la tortuga”, the Spanish word for [MT(“tortuga”) → turtle] turtle.

Toolformer. **Self-supervised** fine tuning.
Generated 25K API calls for each tool (5 tools)

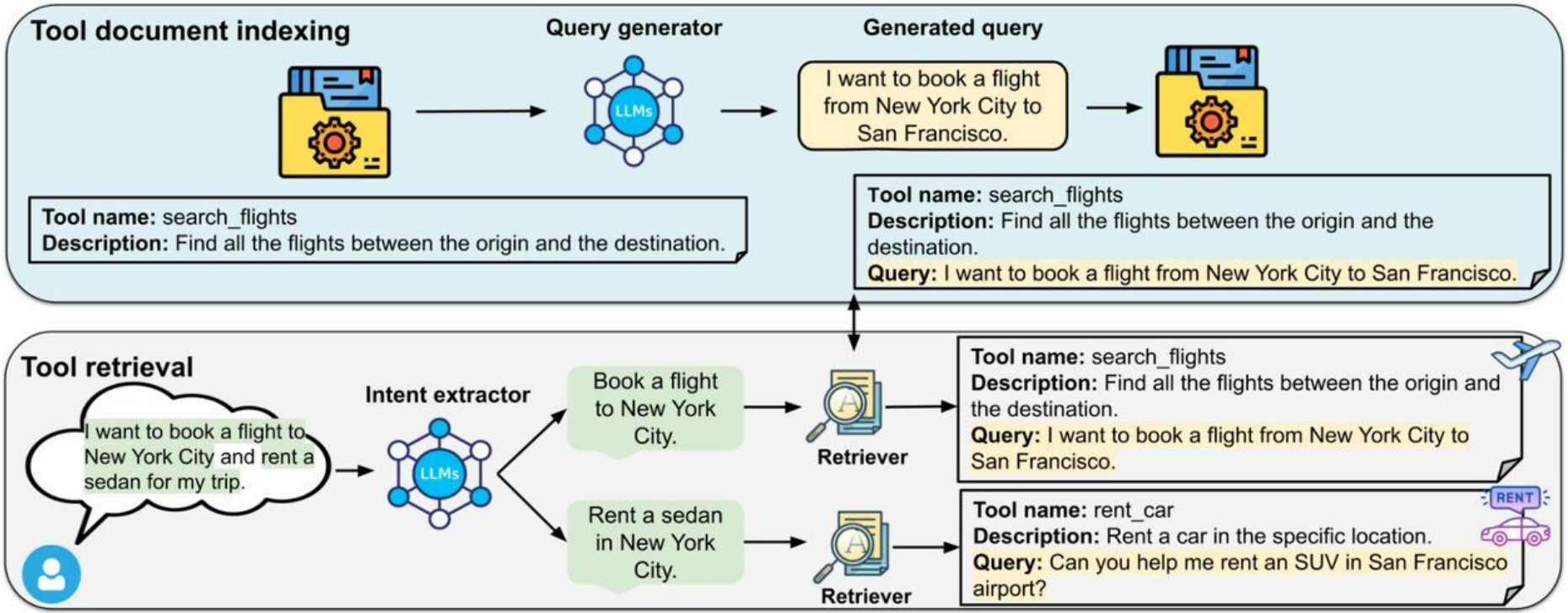
4 **Keep the number of functions small for higher accuracy.**

- **Evaluate your performance** with different numbers of functions.
- **Aim for fewer than 20 functions** at any one time, though this is just a soft suggestion.

https://platform.openai.com/docs/guides/function-calling#best-practices-for-defining-functions

Complex tool docs, huge pool of tools, constantly evolving

Tool retrieval



Complex tool docs, huge pool of tools, constantly evolving

Adding <think></think> tokens (curated/synthetic with o1, R1…)

<tools>

[{'type': 'function', 'function': {'name': **'search_pubmed'**, 'description': 'Search for scientific articles in PubMed database', 'parameters': {'type': 'object', 'properties': {'query': {'type': 'string', 'description': 'The search query for PubMed'}, 'max_results': {'type': 'integer', 'description': 'Maximum number of results to return (default: 5)'}, 'sort_by': {'type': 'string', 'enum': ['relevance', 'date', 'citations'], 'description': 'How to sort the results'}}, 'required': ['query']}}},

{'type': 'function', 'function': {'name': **'analyze_molecule'**, 'description': '**Analyze molecular structure and properties**', 'parameters': {'type': 'object', 'properties': {'molecule_id': {'type': 'string', 'description': 'Molecule identifier (PubChem CID, SMILES, or InChI)'}, 'properties': {'type': 'array', 'items': {'type': 'string', 'enum': ['3d_structure', 'pharmacokinetics', 'binding_affinity', 'toxicity']}, 'description': 'Properties to analyze'}}, 'required': ['molecule_id']}}},

{'type': 'function', 'function': {'name': **'analyze_dataset'**, 'description': '**Perform statistical analysis on scientific datasets**', 'parameters': {'type': 'object', 'properties': {'dataset_url': {'type': 'string', 'description': 'URL of the dataset to analyze'}, 'analysis_type': {'type': 'string', 'enum': ['descriptive', 'regression', 'correlation', 'clustering'], 'description': 'Type of analysis to perform'}}, 'required': ['dataset_url', 'analysis_type']}}}]
</tools>

user Can you find recent research papers on the effects of curcumin on inflammation?

<start_of_turn>model
<think>The user is asking about recent research papers on curcumin and inflammation. This is clearly a scientific literature question, and the most appropriate tool for this would be the search_pubmed function.

Let me think about how to structure this query:

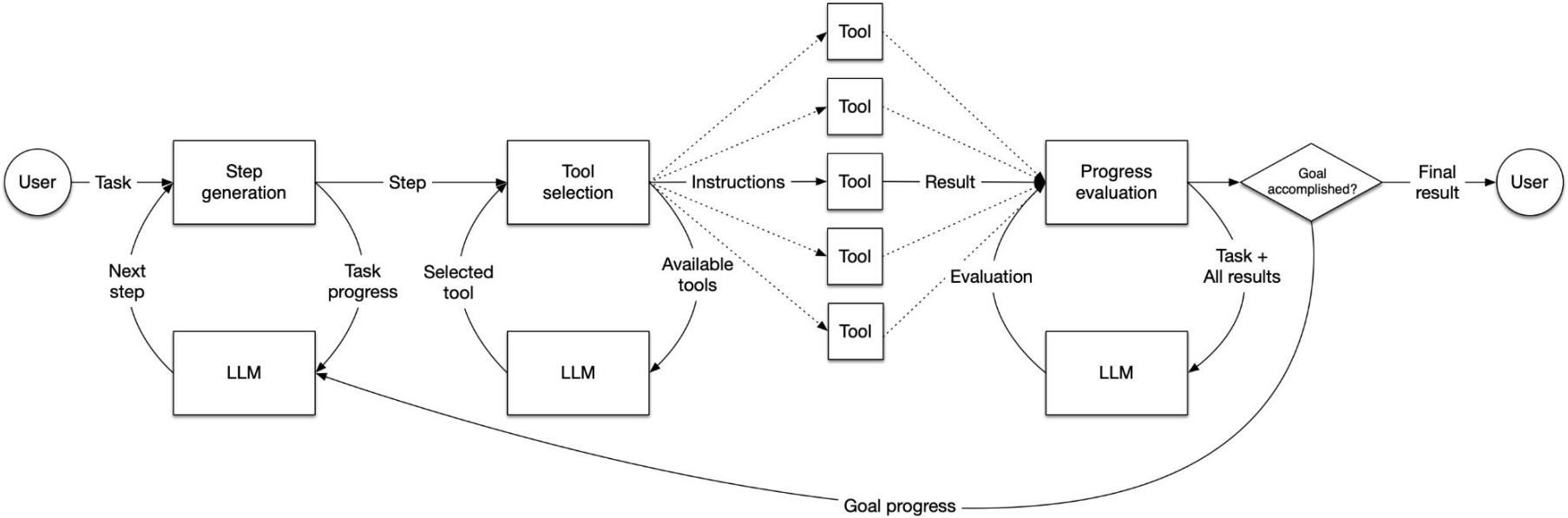
- The main search terms would be "curcumin" and "inflammation"
- Since they asked for "recent" research, I should probably sort by date
- A reasonable number of results might be 5, which is the default, but I'll specify it anyway for clarity
- The query should combine these terms, perhaps: "curcumin inflammation"

I'll use the search_pubmed function with these parameters to find the most relevant and recent papers on this topic.
</think>

<tool_call> {"name": "search_pubmed", "arguments": {"query": "curcumin inflammation", "max_results": 5, "sort_by": "date"}} </tool_call><end_of_turn><eos>

<start_of_turn>tool <tool_response> {"results": [{"title": "Curcumin as a potential therapeutic agent for rheumatoid arthritis: Latest evidence on its molecular mechanisms", "authors": "Wang J, Zhang C, He W, Gou X", "journal": "Int Immunopharmacol", "publication_date": "2023-11-15", "doi": "10.1016/j.intimp.2023.110631", "abstract":....

MetaTool Benchmark for Large Language Models: Deciding Whether to Use Tools and Which to Use


User
Task
Step generation
Step
Tool selection
Instructions
Tool
Tool
Tool
Tool
Tool
Result
Progress evaluation
Goal accomplished?
Final result
User
Next step
Task progress
Selected tool
Available tools
Evaluation
Task + All results
LLM
LLM
LLM
Goal progress

llamaindex